Beyond prompts: context architecture for *reliable* LLM systems.

The system around the model — what it knows, what it should ignore, and how it recovers.

SPEAKER

Chrys Li

AI systems & service design · 7modes

SOURCE

The system around the model

June 2026 · 7modes.com

I

ACT ONE OF FIVE

THE QUESTION I KEEP GETTING ASKED

“How do we *prompt* it so it behaves the way we want?”

A reasonable question — the prompt is the part you can read, edit, and argue with. And far too small a place to look.

THE VISIBLE SURFACE

The prompt is the *visible* surface. Behaviour is shaped by everything *behind* it.

A PROMPT IS

An *instruction.*

Static. Defined once. Useful for tone, role, output expectations, behavioural guidance. Still matters — just carries less weight.

vs

A WORKFLOW IS

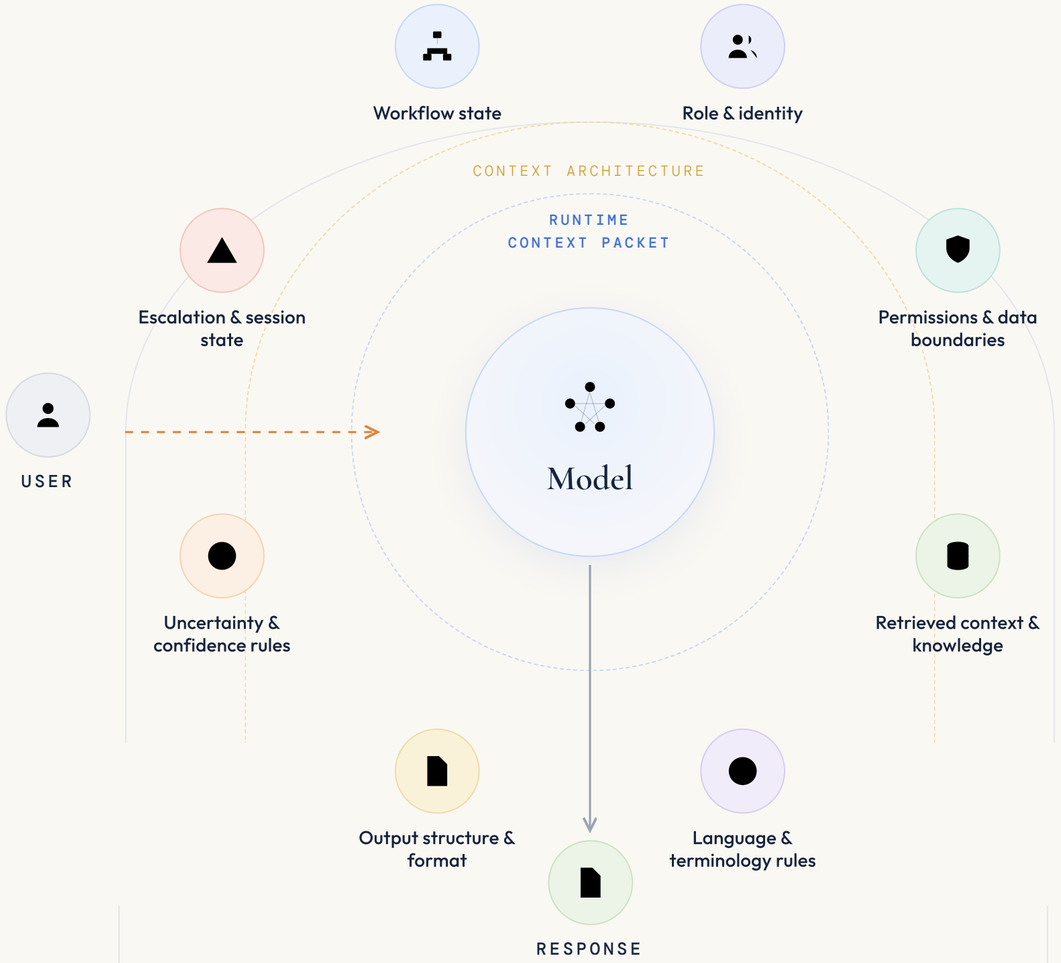
A *stateful* operating environment.

Dynamic. Reassembled per turn. Defines what the model receives when it is actually asked to act.

I

THE WHOLE PICTURE

The model is *not* the system.



II

ACT TWO OF FIVE

THE MISSING LAYER

There is a layer almost nobody *designs on purpose.*

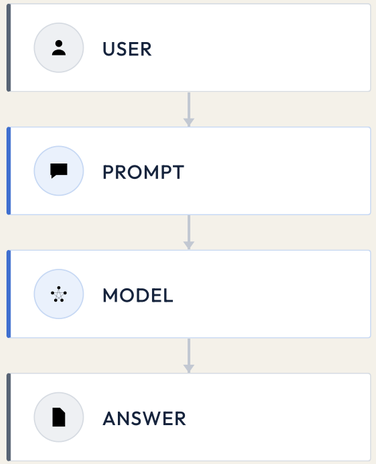
Between the prompt and the platform sits the layer that decides what the model knows.

2

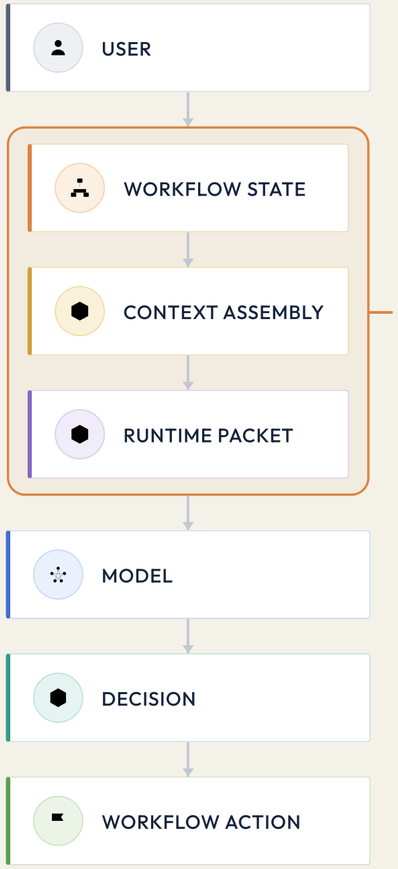
SAME MODEL · DIFFERENT ENVIRONMENT

The missing *layer.*

SIMPLE CHATBOT



WORKFLOW INTELLIGENCE



THE MISSING LAYER

Absent in a simple chatbot.

The system provides the conditions. The model produces the behaviour the workflow needs.

III

ACT THREE OF FIVE

WHAT TEAMS SHOULD BUILD

Build the *environment*, not just the prompt.

Context architecture is a build artifact — versioned, tested, reviewed.

THE FRAMEWORK

Five layers. *Reviewable* on their own. *Aligned* with each other.

Each layer answers a different kind of system question. The work ships only when each layer is consistent with the others. Alignment is the design product.

00

SYSTEM OVERVIEW

Purpose & *principles*

Purpose. Principles that guide behaviour.

01

BEHAVIOURAL SPECS

How it should *act*

When to ask, infer, retrieve, escalate.

02

CONTEXT ARCHITECTURE

What the model *knows*

What context exists, when, how it is read.

03

TESTING & DIAGNOSTICS

How it is *tested*

Which layer failed when the answer is wrong.

04

UX & SERVICE DESIGN

How it *shows up*

Stages. Handoffs. Review. Recovery.

THE EXAMPLE I'LL CARRY THROUGH THE TALK

A public-service *triage assistant.*

RUNNING EXAMPLE

USERS

Residents · Businesses · Internal service teams

INPUT TYPES

Question · Complaint · Service request · Issue · Feedback

OUTPUT

Structured case · Routed to the right team

WHAT THE SYSTEM ACTUALLY HAS TO DO

- Identify *missing information* — only what affects routing or urgency
- Distinguish a *question* from a *complaint* from a *service request*
- Preserve the user's *original intent* before restructuring the case
- Check related case history *only when permitted*
- Prepare a structured case record and route to the right team

WHY IT'S HARD

Multilingual · multi-tenant · permissioned · interruptible.

None of that fits in a prompt.

LAYER 00

00

OF FIVE · SYSTEM OVERVIEW

Purpose and principles. The slogan filter — if you can't test it, it isn't a principle yet.

System *overview.*

Operational principles a team can defend — and write a test for.

P · 01	Reduce user effort — *keep important decisions visible.*
P · 02	Ask *fewer, better* questions.
P · 03	Preserve user intent when *restructuring* content.
P · 04	Separate facts, retrieved information, inference, and *uncertainty.*
P · 05	Protect restricted information — *even when disclosure would feel convenient.*

LAYER 01

01

OF FIVE · BEHAVIOURAL SPECIFICATIONS

How the assistant should act inside a specific role
— before the role is compressed into a prompt.

Behavioural *specifications.*

For the triage assistant, the spec answers six questions.

B · 01	When should it *ask?*
B · 02	When should it *infer?*
B · 03	When should it *retrieve?*
B · 04	When should it *escalate?*
B · 05	How should it handle *incomplete input?*
B · 06	How should it *preserve user intent?*

LAYER 02 · CORE

02

OF FIVE · CONTEXT ARCHITECTURE

The operational layer. In most teams I work with, the weakest. The single largest leverage point for reliability.

Context architecture is a *designed artifact.*

Not an accidental side effect of how the code happens to be wired. The layer defines what context exists, where it comes from, when it is included, how it is prioritised, and how the model is told to interpret it.

DECIDES

- what is *always* included
- what depends on *role*
- what depends on *step*
- what is *retrieved dynamically*

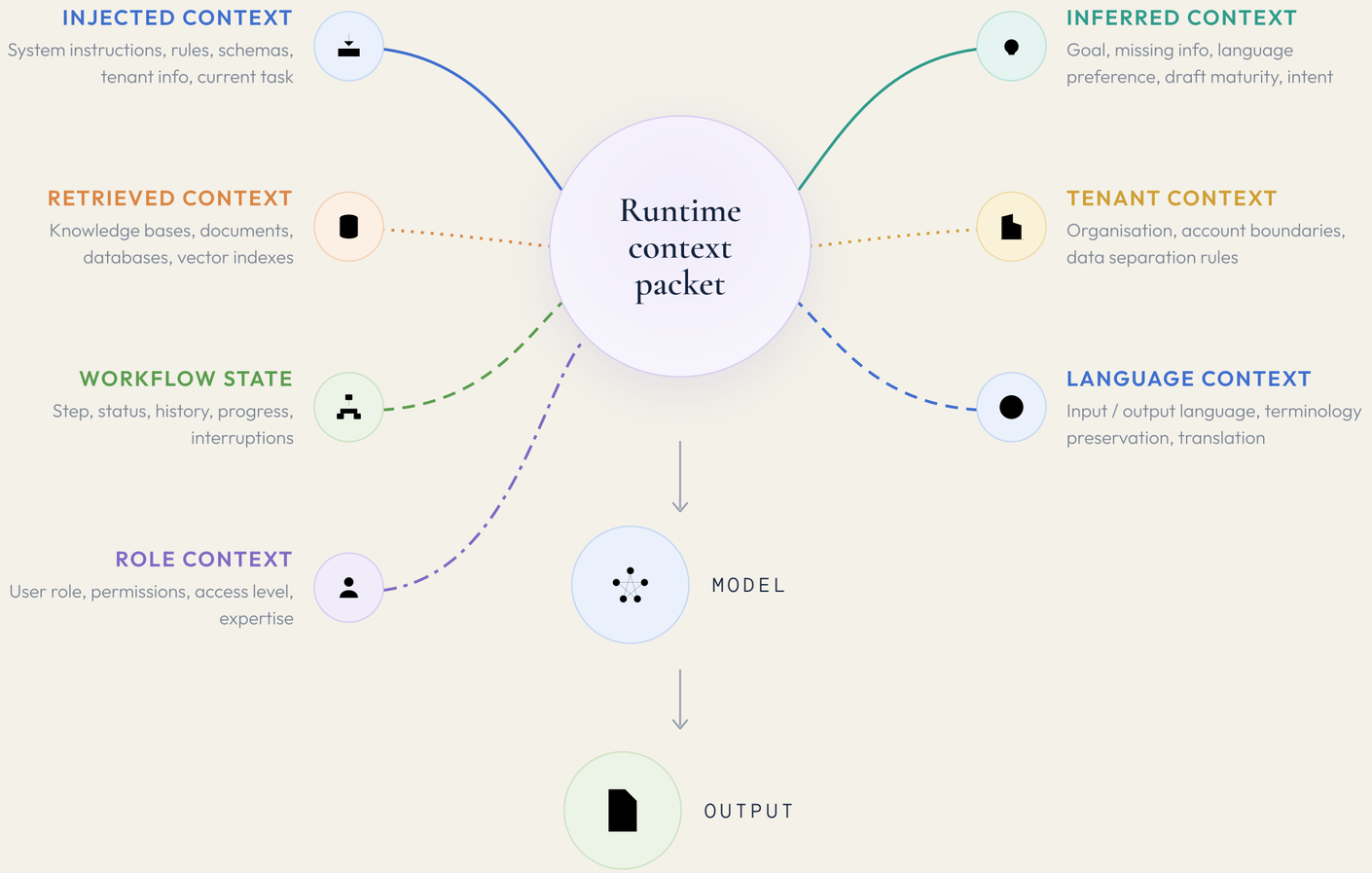
ALSO DECIDES

- what is *inferred* from the session
- what is *restricted* from model access
- how *conflicts* are handled
- how context is *labelled*

3

SEVEN SOURCES · ONE PACKET

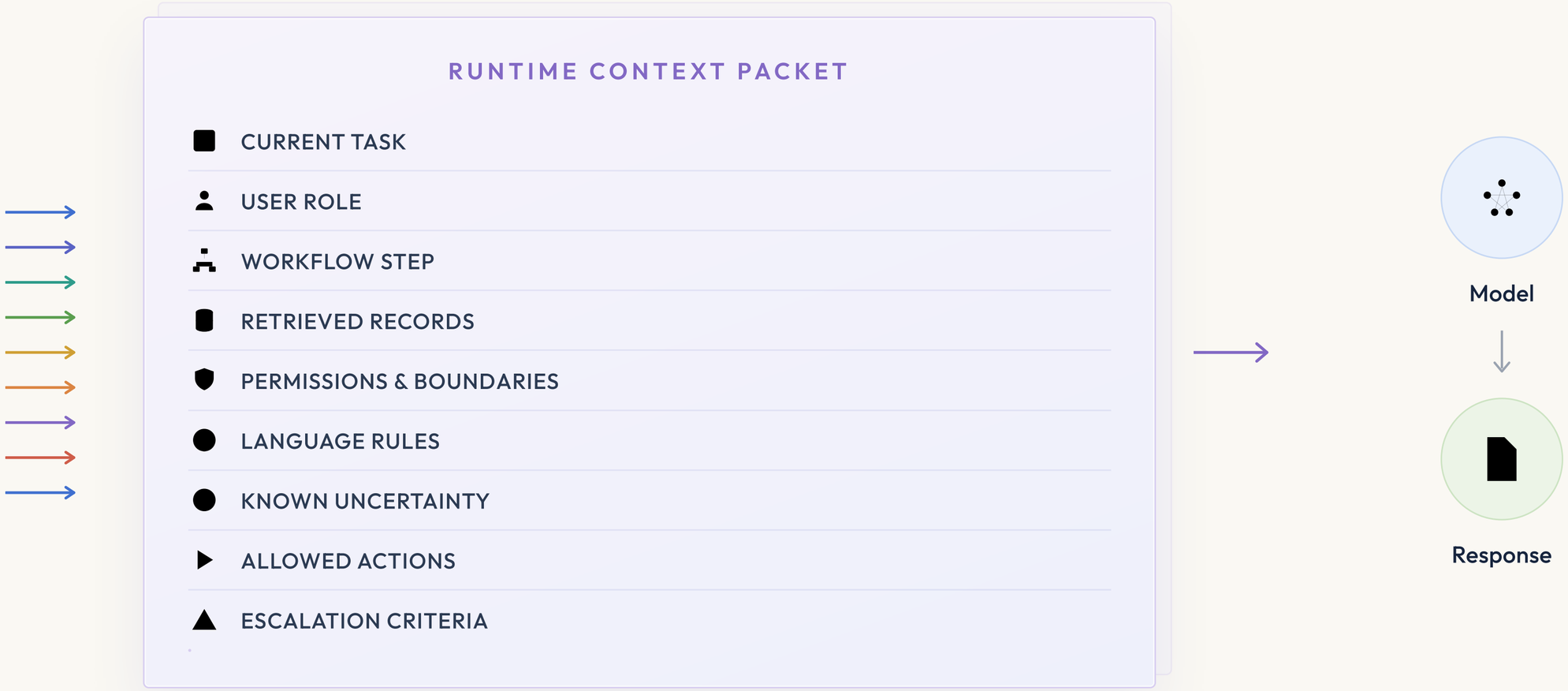
Context is *assembled,* not discovered.



4

WHAT THE MODEL RECEIVES THE MOMENT IT ACTS

The runtime context *packet.*



The model does not see the world. It sees the packet.

OPERATING FRAME · LAYER 02

A runtime context *packet.*

The structured object the model receives at the moment it is asked to act.

The point is not that the model reads every field. *You* decided what is in it and what isn't.

The prompt sits inside this. It carries less unsupported weight.

```
// runtime_context.json
{
  "task":            "triage_incoming_request",
  "workflow_step":   "clarifying_user_intent",
  "user": {
    "role":          "resident",
    "tenant":        "city_of_anywhere",
    "locale":        "en-US"
  },
  "known_facts":     [ ... ],
  "retrieved": [
    { "id": "case_4471", "source": "case_history" },
    { "id": "policy_22", "source": "service_kb" }
  ],
  "inferred":        { "likely_intent": "file_complaint" },
  "restricted":      [ "internal_notes", "other_tenants" ],
  "output_schema":   "case_record_v3",
  "allowed_actions": [ "ask", "classify", "draft_case" ],
  "escalate_if":     [ "safety_concern", "out_of_scope" ]
}
```

ASSEMBLY RULES · LAYER 02

Decide it *once.*

Write it *down.*

Assembly rules turn context from a coincidence into a design decision. They are the part of the system that survives a model swap, a vendor change, and a new engineer joining the team.

RULE TYPE 01

Inclusion

Always-in. Conditional on role. Conditional on step.

RULE TYPE 02

Priority

Which sources win when two of them disagree.

RULE TYPE 03

Labelling

Fact. Retrieval. Inference. Uncertainty. Source attribution.

RULE TYPE 04

Compression

When the packet is too large, what gets summarised, dropped, deferred.

THE REFRAME

Assembly is a set of *gates* — and the prompt is one field *inside.*

BEFORE

Prompt on *top* of the system.

Asked to carry tone, rules, state, permissions, schema — everything.

→

AFTER

Prompt *inside* the packet.

Gates decide inclusion, priority, labelling, conflict, compression. The prompt just carries tone & phrasing.

Make the gates explicit and assembly becomes a design artifact — not a side effect of how the code got wired.

IV

ACT FOUR OF FIVE

RELIABILITY COMES FROM DIAGNOSIS

A bad answer is a *system* failure.

Once the packet exists, you can stop grading outputs and start locating causes.

LAYER 03

03

OF FIVE · TESTING & DIAGNOSTICS

The point is not whether the answer was good. The point is whether the system behaved correctly given the context it received.

“Sounds good” is not a QA methodology.

WEAK TEST	*Was this answer good?*
STRONGER TEST	*Did the system behave correctly given the context it received?*

The second framing connects output quality back to system conditions — and lets you tell prompt failures, retrieval failures, packet assembly failures, and permission failures apart.

FAILURE MODES · LAYER 03

Failure modes you can *name* — and *test.*

Each one has a different cause. Each one has a different fix. The first job of diagnostics is to tell them apart.

F · 01

Workflow *drift*

Forgets which step the user is on. Behaves like the first turn every turn.

F · 02

Retrieval *misuse*

Over-weights irrelevant retrieved content because it appears in the packet.

F · 03

Over-*questioning*

Asks for information already available. Asks questions that do not improve the outcome.

F · 04

Hallucinations

Invents workflow rules, routing logic, escalation explanations, case details.

F · 05

Language *instability*

Behaviour changes when the user switches languages. Translates terms that should not change.

F · 06

Permission *leakage*

Reveals restricted records through summaries, similarity, or indirect references.

F · 07

False *related-case* matches

Flags unrelated records as related. Misses relevant history when wording differs.

F · 08

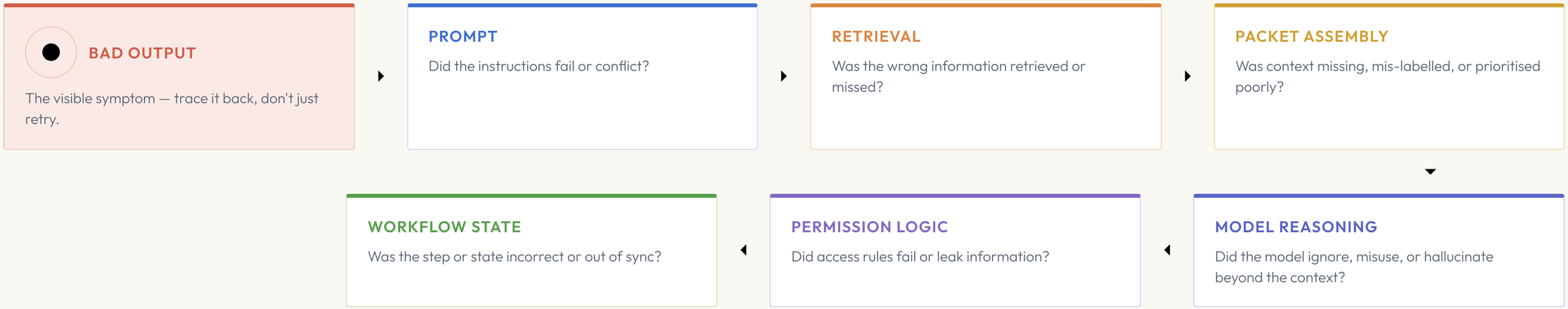
Confidence *collapse*

Treats inferred context as fact. Cannot surface its own uncertainty.

6

WHEN IT BREAKS, ASK WHICH LAYER

Failures are rarely the *model* alone.



FAILURE · PERMISSION LEAKAGE · LIVE

Watch it *happen.*

RESIDENT

"Has anything like my issue happened on my street before?"

ASSISTANT

"Yes — a similar case at 14 Oak St was resolved last month. The internal note says the resident there, J. Alvarez, had filed three prior complaints."

Fluent. Helpful. Leaks a restricted internal note — *exactly what "sounds good" waves through. The packet marked it restricted;* permission logic never enforced it.

V

ACT FIVE OF FIVE

THE LAST LAYER

Where the system *shows up.*

Service design and UX sit on top of the framework — and only pay off when they're aligned with everything beneath them.

LAYER 04

04

OF FIVE · UX & SERVICE DESIGN

Entry points, conversation stages, review moments, handoffs, interruption paths, recovery.

Alignment is the *point.*

IF	*the UX asks the assistant to do something the context architecture does not support …*
THEN	*the system drifts.*
IF	*the architecture supports a capability the UX never exposes …*
THEN	*the value stays hidden.*

THE FULL FRAMEWORK

Five layers. *One* system.

We've now walked all five. The work ships only when each layer is consistent with the others. Alignment is the design product.

00

SYSTEM OVERVIEW

Purpose & *principles*

Purpose. Principles that guide behaviour.

01

BEHAVIOURAL SPECS

How it should *act*

When to ask, infer, retrieve, escalate.

02

CONTEXT ARCHITECTURE

What the model *knows*

What context exists, when, how it is read.

03

TESTING & DIAGNOSTICS

How it is *tested*

Which layer failed when the answer is wrong.

04

UX & SERVICE DESIGN

How it *shows up*

Stages. Handoffs. Review. Recovery.

A NOTE ON REVIEW

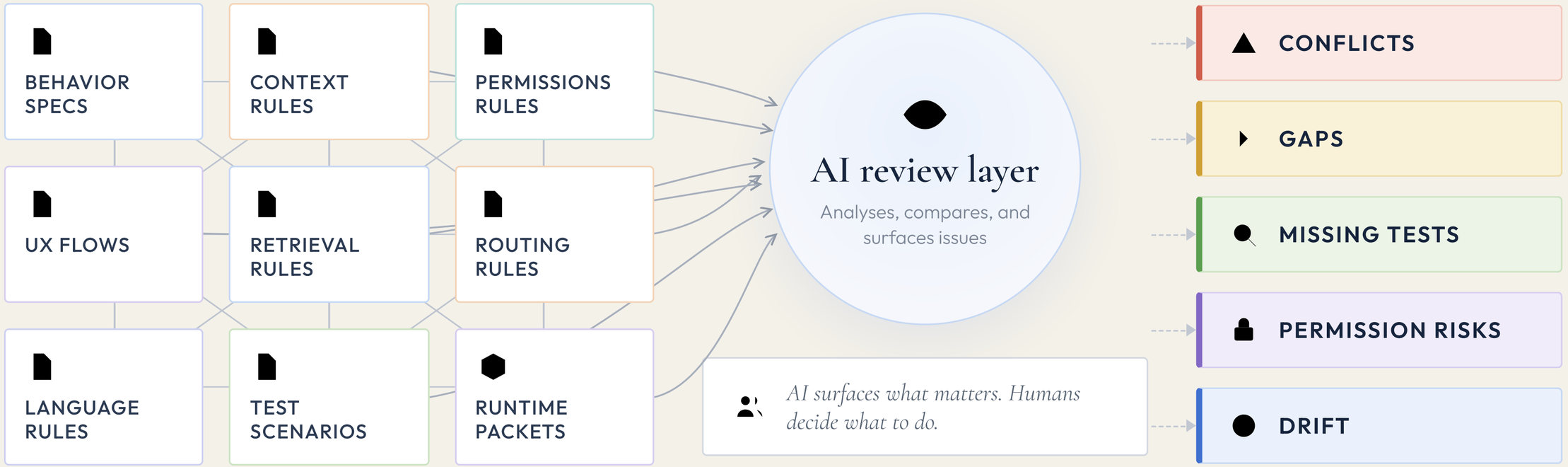
Human review does *not* *scale* cleanly.

Each document may be clear in isolation. They need to *work together.* That cross-layer review is itself becoming a job for AI-assisted review layers.

7

TOO COMPLEX TO REVIEW BY HAND

The architecture *reviews itself.*



THE WHOLE TALK IN TWO LINES

The model is *important.*

The system around the model determines what it can *reliably do.*

That system is the work.

Q & A · 5 MINUTES

Thank *you.*

ARTICLE

The system around the model

7modes.com · Signals

TEMPLATES & SAMPLE ARCHITECTURE

github.com/chrysli/
AI-Behavior-Context-Architecture-Framework

SCAN FOR REPO & SAMPLES

CONTACT

Chrys Li · chrysli@7modes.com

SCAN FOR LINKEDIN